INTRODUCTION TO THE INTERNET OF THINGS · LAB 3

Lab 3 — Getting Online: Wi-Fi + MQTT

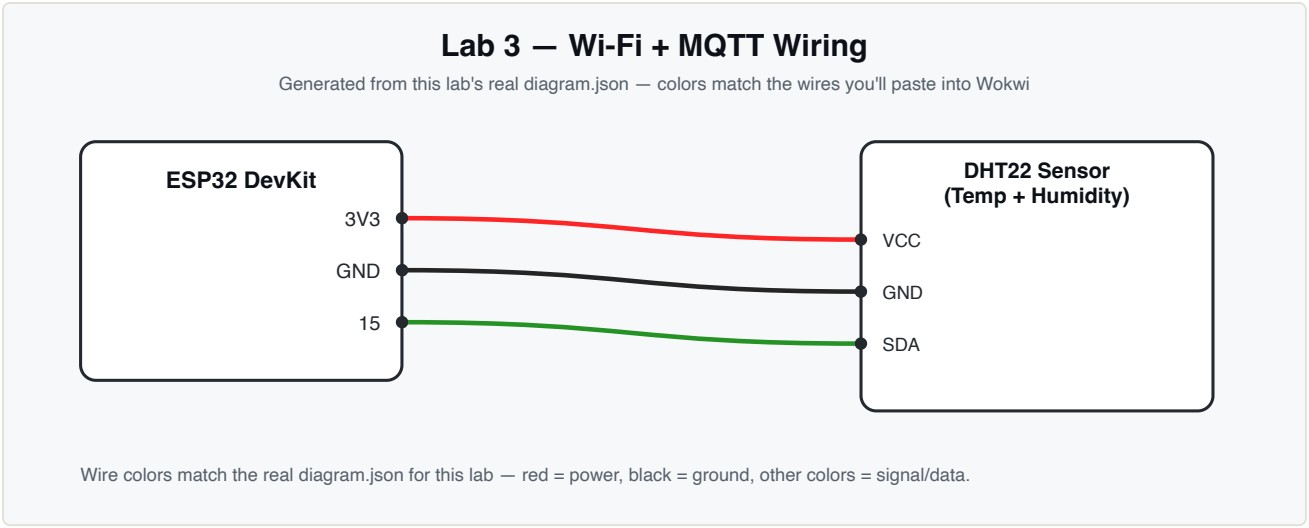
Time: ~23 minutes in the Telecom & Data-Centre Edition (~30 minutes on your own) **You'll learn:** how to connect a device to the internet, and how to publish live data using MQTT — the messaging protocol most real IoT systems run on.

> **Telecom & Data-Centre Edition**
>
> **The story:** The tower site reports its shelter readings to the network operations centre (NOC) over the network, using MQTT.
>
> **Your team:** Teams of three. Rotate the roles from the last lab: **builder** (keyboard), **analyst** (reads the guide, checks the output), **decision owner** (answers the manager task).
>
> **Manager task:** Who should get which alert? Think of the NOC, the field engineer on call and the site owner.
>
> **Extension (fast teams):** Give every team's site its own topic, then watch all the sites at once. In `sketch.ino`, change the temperature topic to:
>
> `cpp String topicTemperature = "iot-workshop/towers/" + String(YOUR_NAME) + "/temp";`
>
> In HiveMQ, subscribe to `iot-workshop/towers/+/temp`. The `+` matches any one level, so you see every team's site.
>
> **Business-case canvas:** Fill in box 3 (how the data travels).

What you're building

The same DHT22 setup as Lab 2, but now the ESP32 connects to Wi-Fi and **publishes** its readings to a public **MQTT broker** — a server whose only job is to receive messages on named "topics" and pass them on to anyone listening. Think of a topic like a radio station: you publish to a station name, and anyone tuned to that station name hears it, instantly.

Wiring diagram

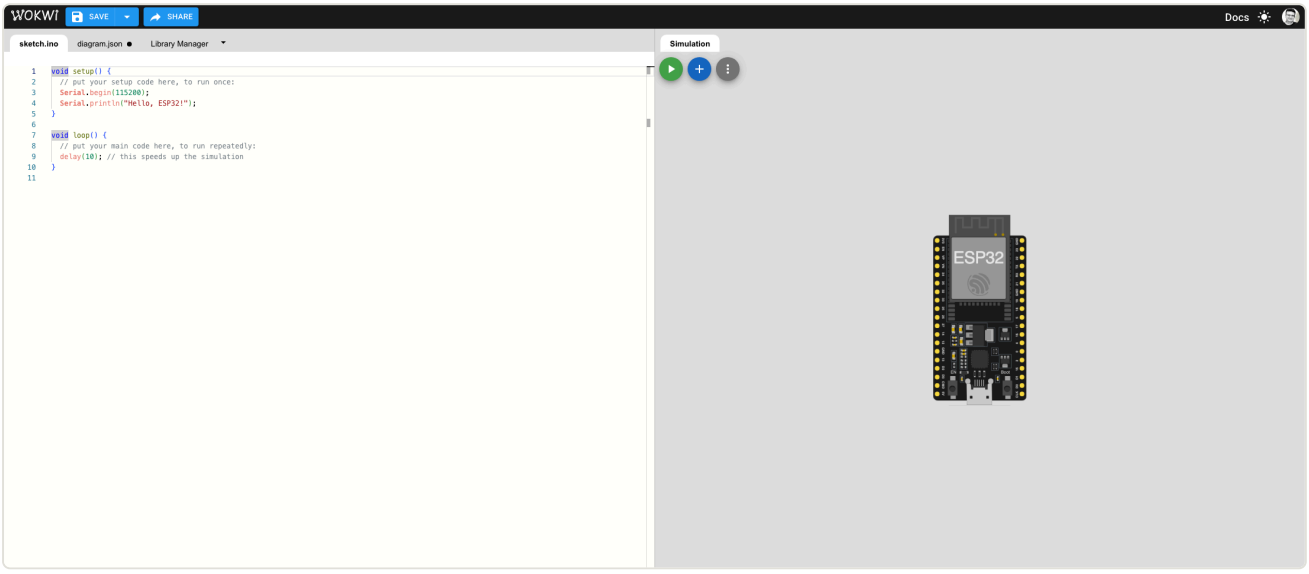


ESP32 Pin	Wire Color	Connects To
`3V3`	red	DHT22 Sensor (VCC)
`GND`	black	DHT22 Sensor (GND)
`15`	green	DHT22 Sensor (data pin)

It's the exact same wiring as Lab 2 — this lab is about what the *code* does with the reading, not new hardware.

Step-by-step

1. Open a new ESP32 project

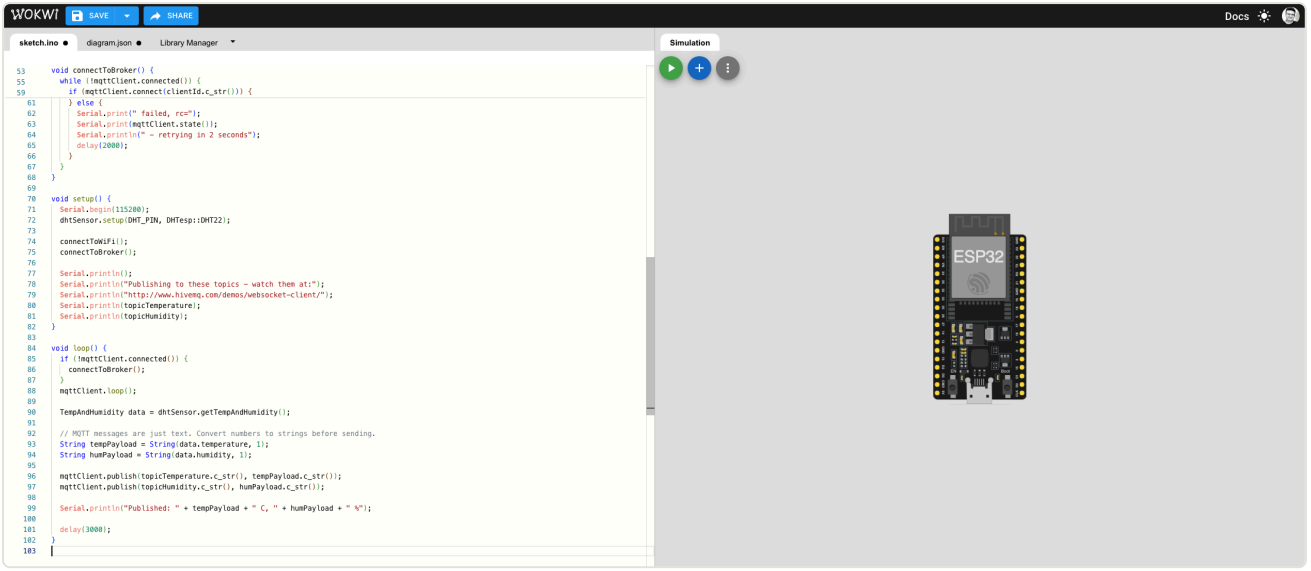


2. Paste the code — but change one line first

Copy `sketch.ino` into Wokwi's **sketch.ino** tab. Before you press Play, find this line near the top:

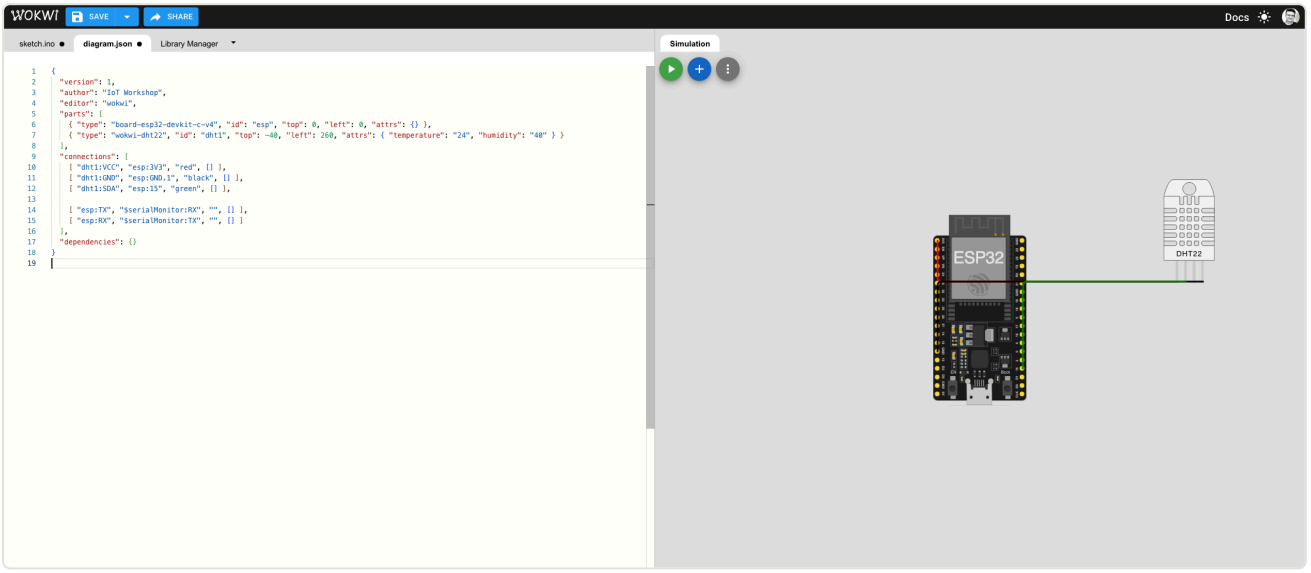
```
const char* YOUR_NAME = "changeme";
```

Change `"changeme"` to something short and unique — your first name is perfect. We're all sharing one public broker in this lab, and topic names need to be unique so your data doesn't mix with your neighbor's.



3. Paste the circuit

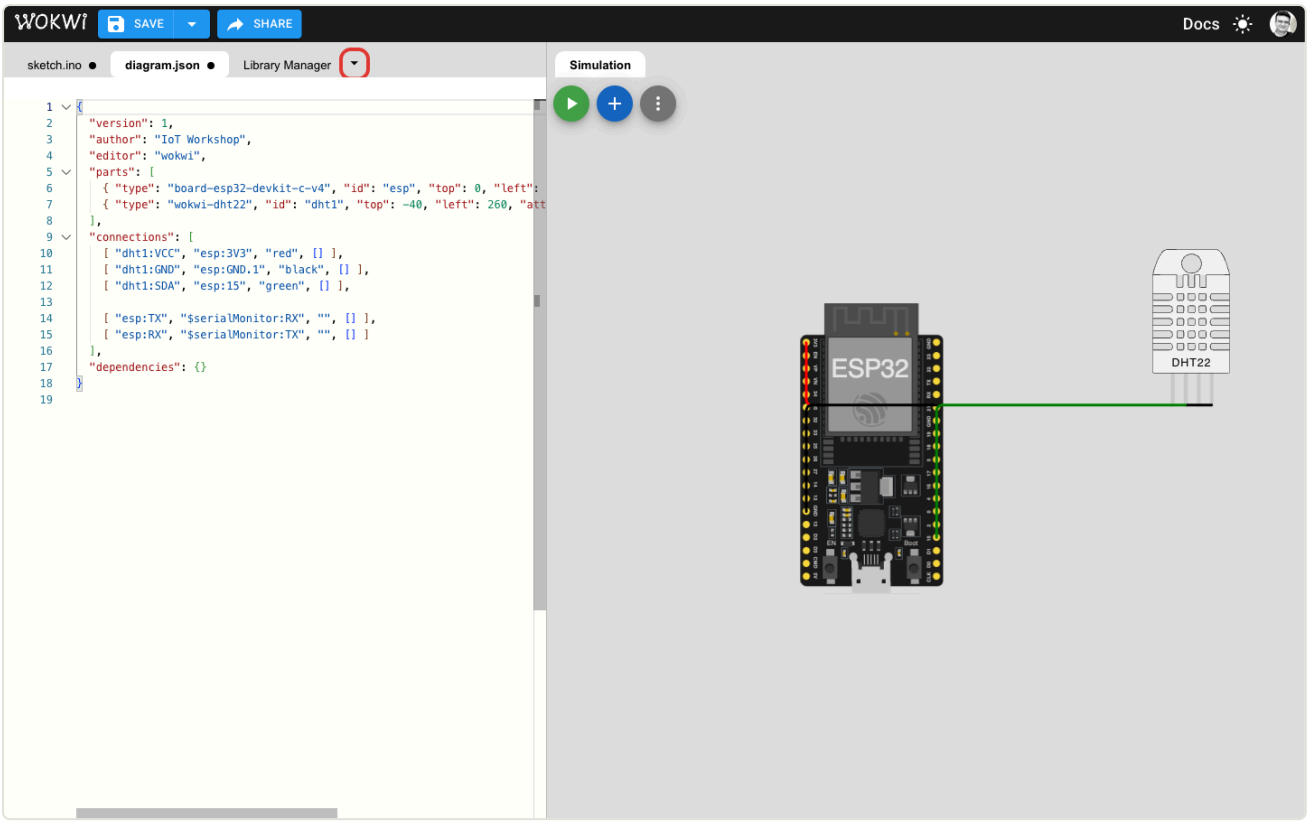
Copy `diagram.json` into Wokwi's **diagram.json** tab.



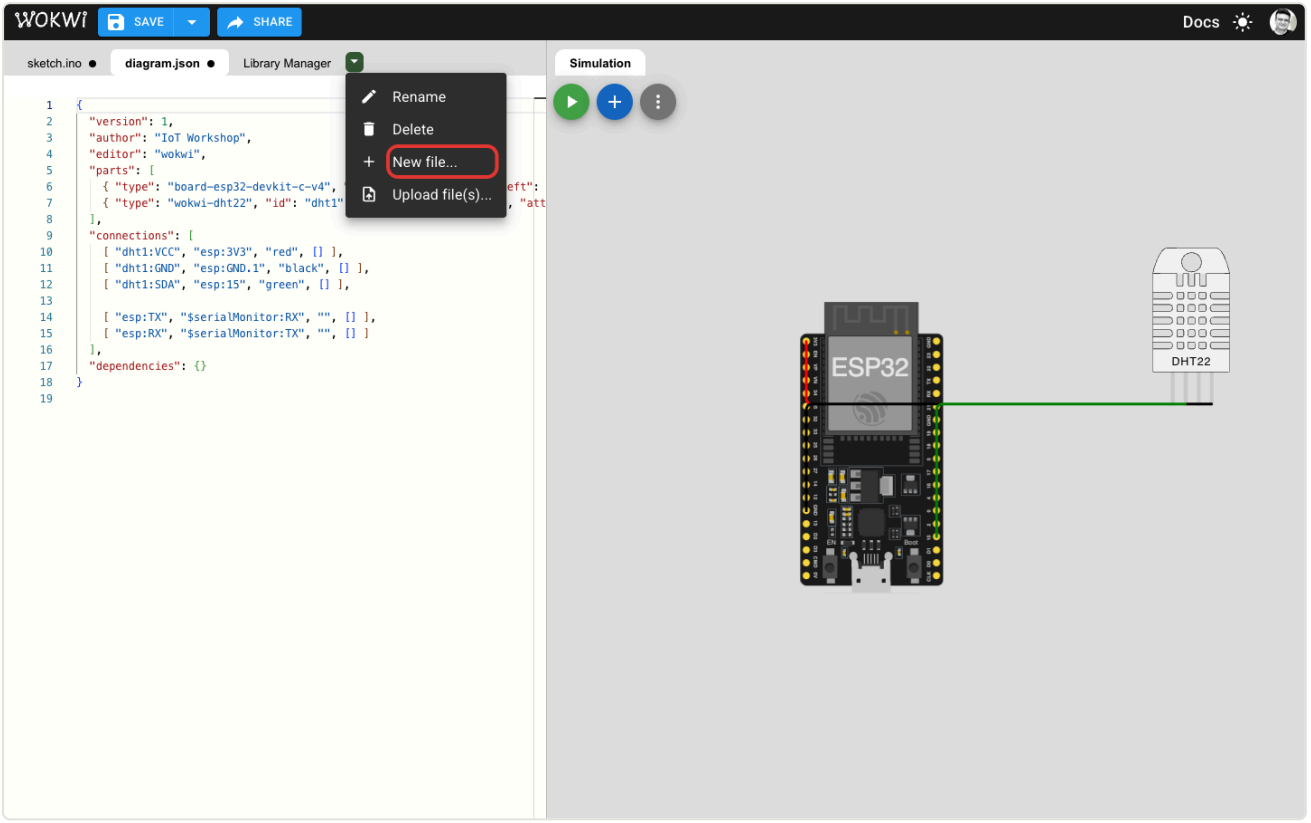
4. Add the library list

This sketch needs two libraries (`DHTesp` and `PubSubClient`).

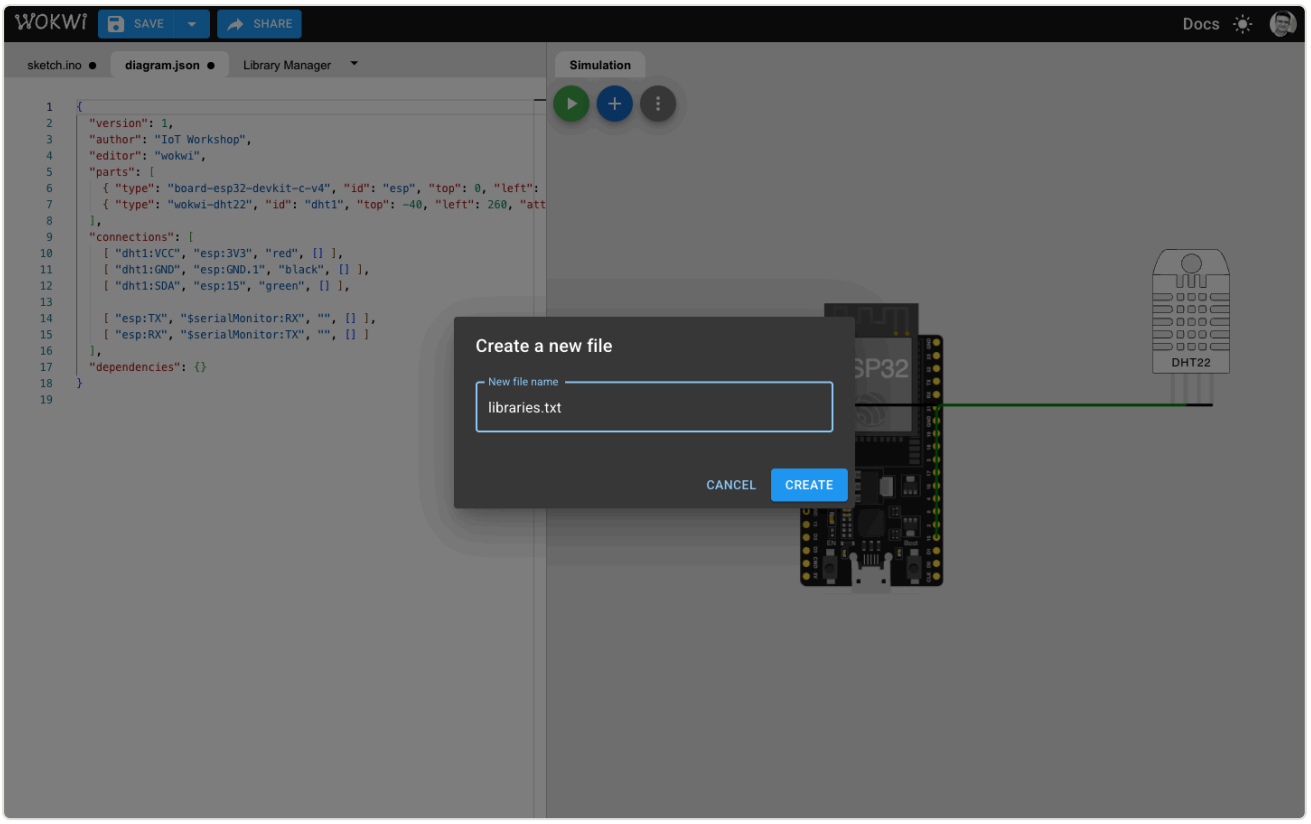
1. Click the small ▾ next to the **Library Manager** tab.



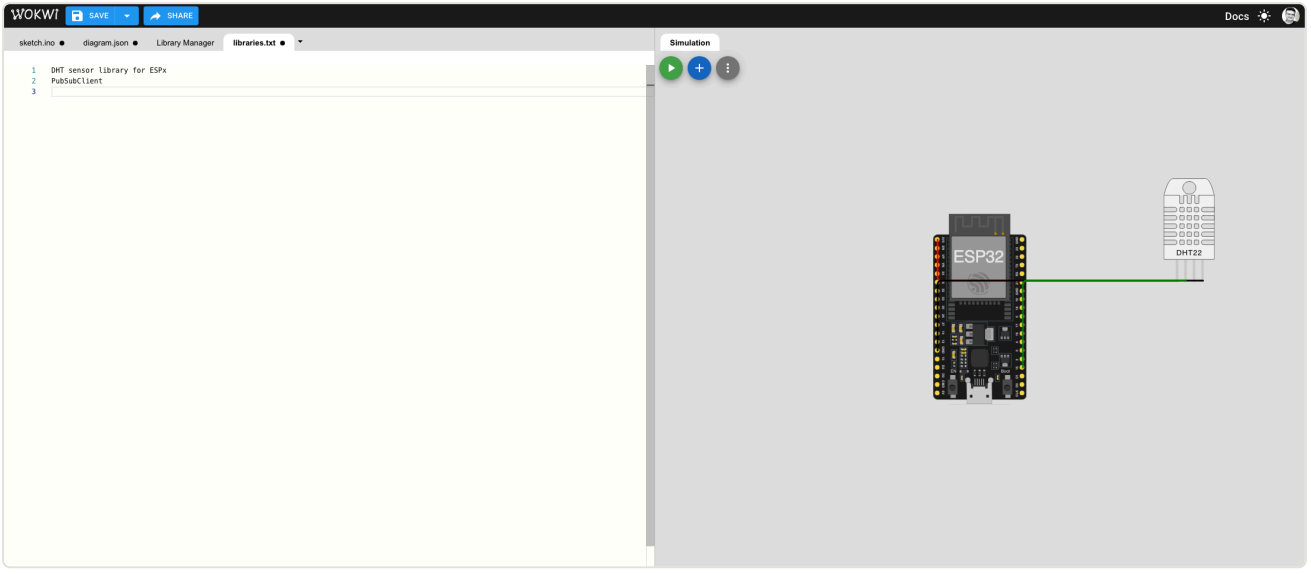
2. Choose **New file...**.



3. Type the name exactly `libraries.txt` and click **CREATE** (or press Enter).



4. Copy the contents of `libraries.txt` from this folder and paste them into the new **libraries.txt** tab.



5. Press Play

Click ▶ **Play**. This is the heaviest compile so far (Wi-Fi + MQTT + sensor code, two libraries) — the first build can take a couple of minutes on Wokwi's free tier, and may show "Build Servers Busy." That's a queue limit, not an error — click Play again if it happens.



Watch the Serial Monitor: it should print `connected!` for Wi-Fi, then `connected!` for the MQTT broker, then start publishing readings every 3 seconds.

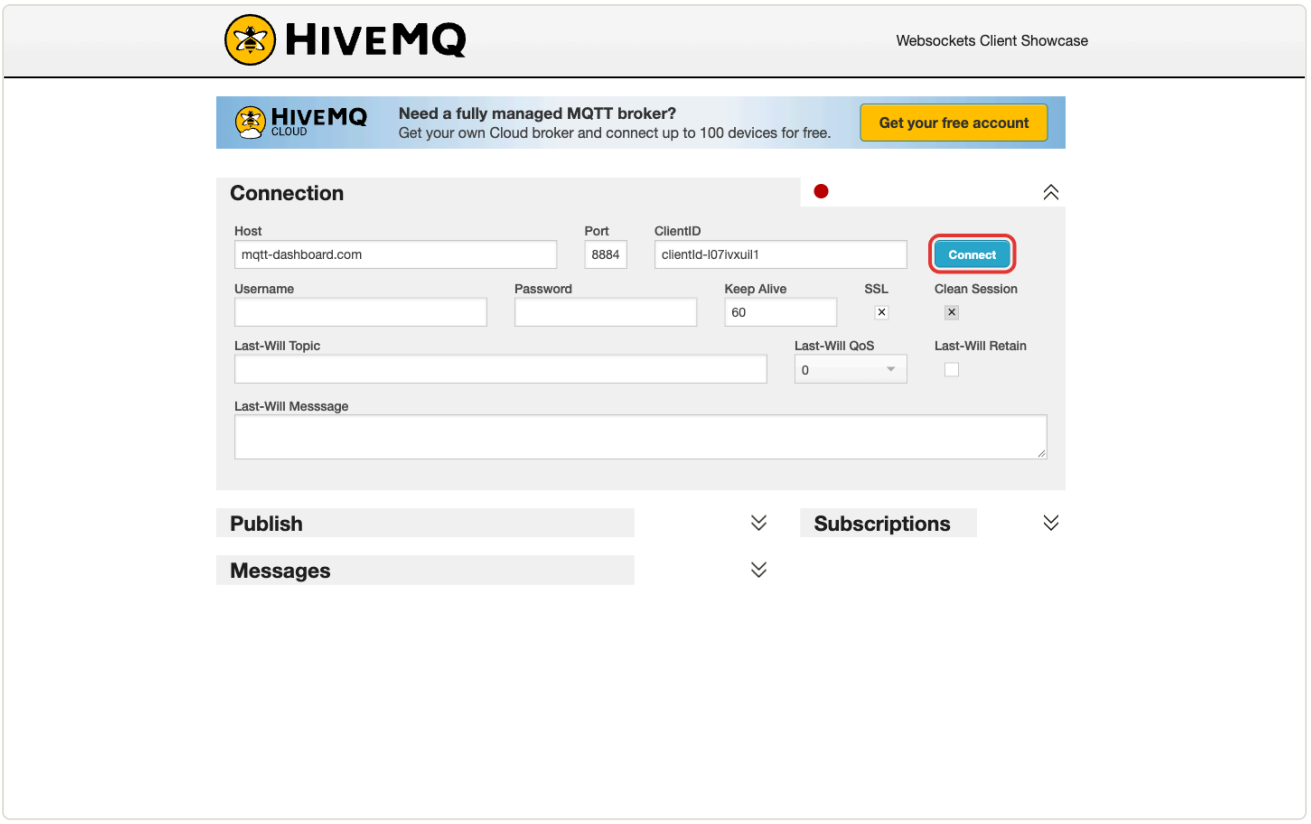
What you should see

```
Connecting to Wi-Fi...... connected!
Device IP address: 10.13.37.2
Connecting to MQTT broker... connected!

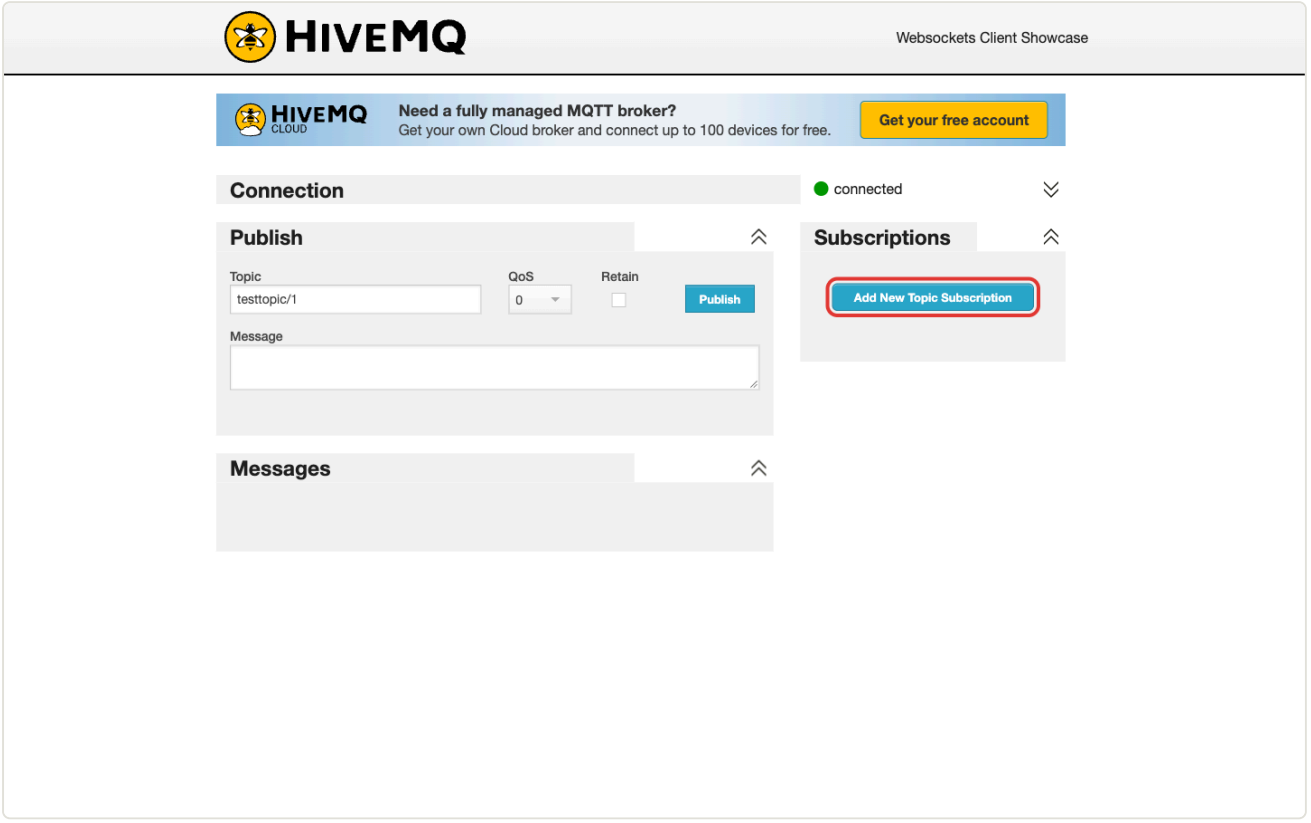
Publishing to these topics - watch them at:
http://www.hivemq.com/demos/websocket-client/
iot-workshop/yourname/temperature
iot-workshop/yourname/humidity
Published: 24.0 C, 40.0 %
Published: 24.0 C, 40.0 %
```

Watch your data arrive, live

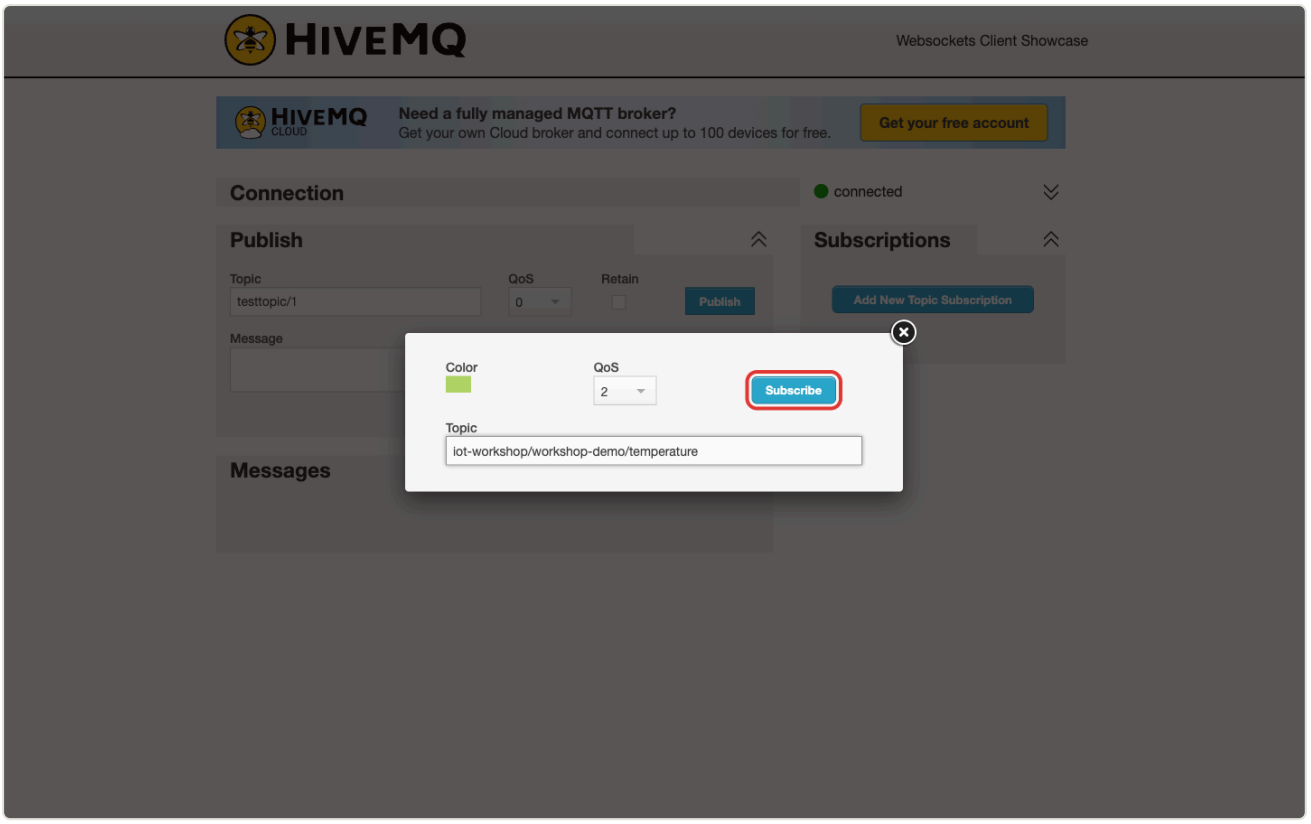
1. On a second browser tab, open the free HiveMQ Web MQTT Client (no account needed), and click **Connect**. The default settings work: it connects to the same HiveMQ public broker your ESP32 is publishing to.



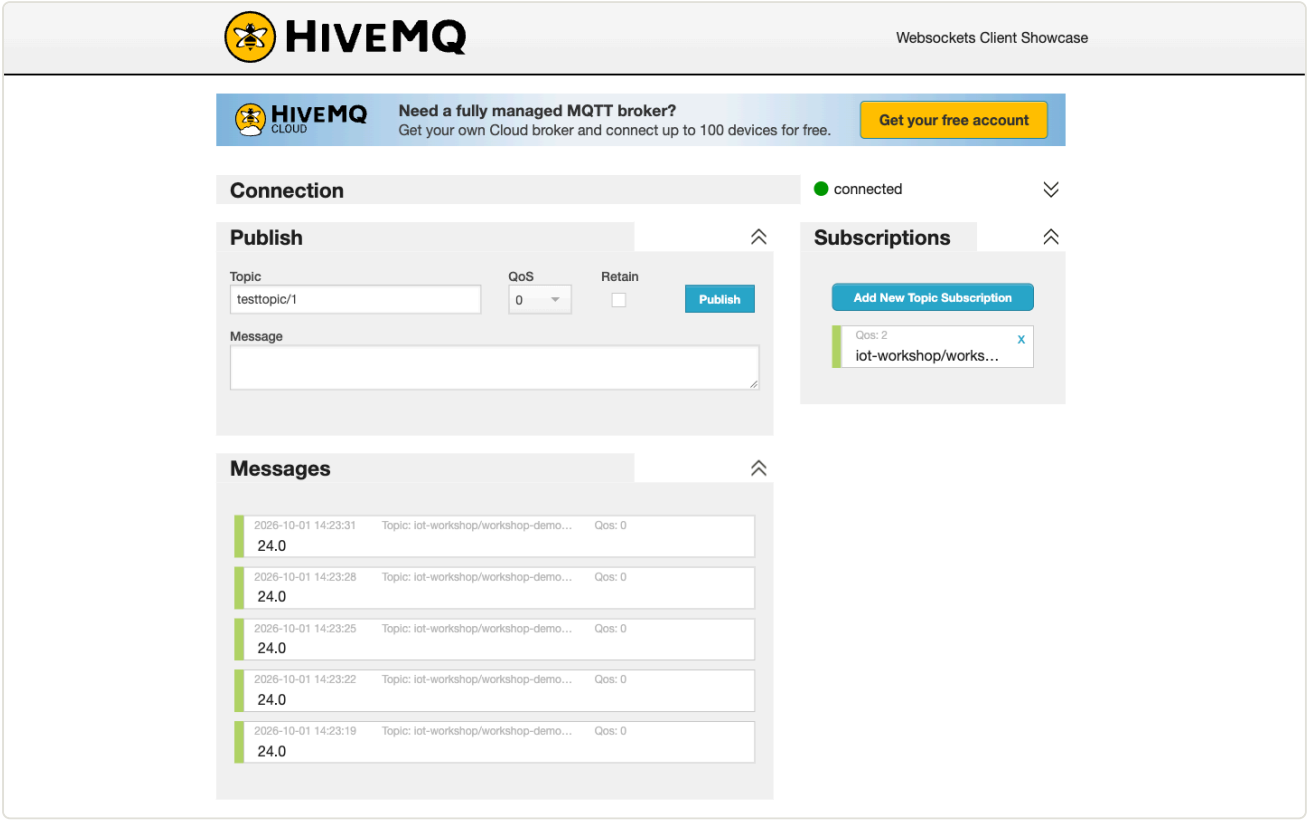
2. The dot next to **Connection** turns green and says **connected**. Under **Subscriptions**, click **Add New Topic Subscription**.



3. In the **Topic** box, type `iot-workshop/YOUR_NAME/temperature` (use the same name you set in the code), then click **Subscribe**.



4. Watch messages appear under **Messages** in real time as your simulated ESP32 publishes them, one every 3 seconds.



Your trainer may also subscribe to `iot-workshop/#` (the `#` wildcard means “everything under this topic”) on the projector, so the whole class’s data appears on one screen at once.

Understand the code

- `WiFi.begin("Wokwi-GUEST", "")` connects to Wokwi's built-in simulated Wi-Fi network — this is the only network name that works inside Wokwi.
- `mqttClient.connect(clientId)` opens a connection to the broker. Every device needs its own unique client ID, which is why the code builds one from your name plus a random number.
- `mqttClient.publish(topic, payload)` sends one message. MQTT messages are always plain text (or raw bytes) — that's why the code converts the temperature number into a `String` first.
- `mqttClient.loop()` must be called often — it's what actually sends and receives data behind the scenes. Forgetting it is a common bug.

A word on security (preview of Module 7)

This broker is **public** — anyone in the world can subscribe to your topic, and anyone could publish fake data to it too. That's fine for a classroom demo, but never acceptable for a real product. Real systems use a private broker with usernames, passwords, and encrypted (TLS) connections.

Troubleshooting

- **`Build failed! ... PubSubClient.h: No such file or directory`.** You skipped step 4, or mistyped a library name in `libraries.txt`. It needs exactly `DHT sensor library for ESPx` and `PubSubClient`, one per line.
- **Stuck on "Connecting to Wi-Fi...".** Very rare in Wokwi (the simulated network is always available), but if it hangs, click Restart — the code will keep retrying forever by design (real Wi-Fi in the field drops sometimes, so the code is written to never give up).
- **MQTT connects but your topic shows nothing in HiveMQ.** Double-check the topic name in step 4 of the HiveMQ instructions matches `YOUR_NAME` *exactly*, including capitalization.

Challenge (if you finish early)

- Publish a single combined message instead of two, formatted like `temperature=24.5,humidity=41.0`, to one topic instead of two.
- Try subscribing (not just publishing) from your ESP32 to a topic, and print any message it receives to the Serial Monitor. Look up `mqttClient.subscribe()` and `mqttClient.setCallback()`.